

Pakistan Law Assistant

A source-grounded RAG assistant for Pakistani law

Project plan and progress report: what has been built in Phases 1-3, how and why,
and the detailed plan for the remaining phases

Author	Raja Rizwan (github.com/rajarizwan2007)
Repository	github.com/rajarizwan2007/pakistan-law-assistant-rag-ai
Date	2 October 2026
Status	Phases 1-3 complete (infrastructure, ingestion, retrieval). Next: Phase 4, answer generation.
Stack	React 19 + TypeScript · Laravel 13 / PHP 8.4 · PostgreSQL 17 + pgvector · Ollama (local LLM) · Docker

Disclaimer: this is a learning and interview-demonstration project. Its output is informational only and is **not legal advice**.

Contents

1. Project overview: the goal, what RAG is, and how a question flows through the system
2. Roadmap at a glance: all phases and their status
3. Phase 1: Development environment (done)
4. Phase 2: Database schema and ingestion of the Pakistan Penal Code (done)
5. Phase 3: Retrieval, relevance threshold and evaluation (done)
6. Phase 4: Answer generation with citations (next)
7. Phase 5: Chat user interface
8. Phase 6: Retrieval improvements
9. Phase 7: More laws: Constitution and Code of Criminal Procedure
10. Phase 8: Answer-quality evaluation
11. Phase 9: Engineering polish, CI and demo
12. Key decisions, how to run the project, and glossary

1. Project overview

Goal. Build a small assistant that answers questions about Pakistani law **only from the text of the law itself**, and shows exactly which section each part of the answer comes from. It is built as a full-time software-engineering learning project and as a portfolio piece to explain in interviews. It is not a production legal service.

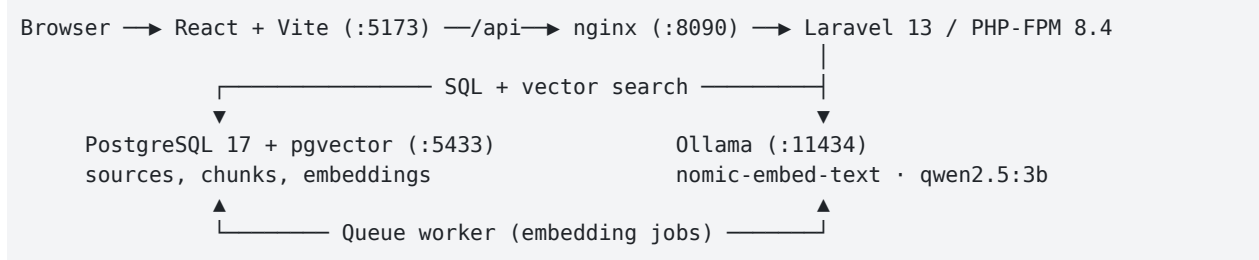
What is RAG? RAG (Retrieval-Augmented Generation) means: first *retrieve* the relevant passages from a trusted collection of documents, then let a language model *generate* an answer using only those passages. This reduces made-up answers (hallucinations) and makes every answer checkable through its citations.

How a question will flow through the system

1. The user types a question in the React app, e.g. “*What is the punishment for theft?*”
2. Laravel turns the question into an **embedding** (a list of 768 numbers that captures its meaning) using the local Ollama model `nomic-embed-text`.
3. PostgreSQL + pgvector finds the law sections whose embeddings are closest in meaning (cosine similarity), using a fast HNSW index.
4. If no section is similar enough (below the **relevance threshold**), the assistant refuses instead of guessing.
5. Otherwise the best sections are numbered [1], [2], ... and given to the local chat model `qwen2.5:3b` with strict instructions: answer only from these sources and cite them.
6. The answer, its citations (e.g. **PPC s.379**) and the source text are shown to the user, with a “not legal advice” disclaimer.

Status today: steps 1–4 work end to end and can be tried in the browser (semantic search with refusal). Steps 5–6, the AI-written answer, are Phase 4.

System architecture



Every box is a Docker container started with one command (`make up`). PHP runs only inside Docker because the host machine has PHP 7.4, while Laravel 13 needs a newer version.

2. Roadmap at a glance

#	Phase	What it delivers	Status
1	Development environment	Docker stack, Laravel 13 API, React app, Postgres + pgvector, Ollama models, health check	✓ Done
2	Schema & ingestion	sources/chunks tables, PDF extraction, section-aware chunking, embeddings, PPC ingested (668 chunks)	✓ Done
3	Retrieval	Retriever service, relevance threshold 0.68, law:search, law:eval (Hit@5 85%), search page	✓ Done
4	Answer generation	Local LLM writes grounded answers with [n] citations, refusal path, POST /api/ask, query log	Next
5	Chat UI	Chat window, clickable citations, source panel, act filter, disclaimer, loading/error states	Planned
6	Retrieval improvements	Legal-term glossary, hybrid keyword + vector search, optional re-ranker, each measured	Planned
7	More laws	Constitution of Pakistan (articles) and Code of Criminal Procedure	Planned
8	Answer evaluation	Measure citation accuracy, faithfulness, refusal accuracy and latency	Planned
9	Polish & delivery	GitHub Actions CI, production Docker build, README screenshots, demo script	Planned

Each phase ends with: tests passing, code-style check passing, documentation updated (docs/architecture.md, docs/progress.md, README), a commit credited only to Raja Rizwan, and a push to GitHub after approval.

Goal: one command starts every service the project needs, the same way on any machine, and a health check proves they are all connected.

Steps completed

1. **Repository set-up.** Connected the local folder to GitHub, renamed the branch to main, added .gitignore (secrets, vendor, node_modules, raw PDFs), README, MIT licence, CLAUDE.md (shared project rules) and CLAUDE.local.md (personal notes, not committed). Switched the remote to SSH because HTTPS had no saved login.
2. **Architecture document.** Wrote docs/architecture.md: services, request flow, database schema, API contract, folder layout, configuration and the reasons behind each choice.
3. **PHP 8.4 Docker image.** docker/php/Dockerfile with the Postgres, intl, zip and bcmath extensions and Composer. It runs as the host user (UID 1000), so files Laravel creates belong to you, not root.
4. **Laravel 13 app** created inside that image, configured for PostgreSQL and Ollama. Laravel's starter AGENTS.md was removed because it told AI tools to install PHP on the host.
5. **Docker Compose stack:** db (Postgres 17 + pgvector), ollama, backend (PHP-FPM), worker (queue), nginx and frontend (Vite). Host ports avoid ones already in use: 5173, 8090, 5433, 11434.
6. **Database init script** creates the pgvector extension and a separate law_assistant_test database, so tests never touch development data.
7. **Health endpoint** GET /api/health checks the database, the pgvector version and that both Ollama models are installed. It has 3 automated tests.
8. **React 19 + TypeScript frontend** (Vite) with a dev proxy for /api (no CORS issues).
9. **Makefile shortcuts:** make up / down / models / migrate / test / artisan / psql / logs / health.
10. **Downloaded the models** nomic-embed-text (274 MB) and qwen2.5:3b (1.9 GB), chosen to run on a CPU without a GPU.
11. **Progress log** docs/progress.md so every session can pick up where the last one stopped.

Result

- Health check is green: database ok (pgvector 0.8.7), both models installed.
- Embedding a sentence takes under 1 second; a short chat answer takes about 7.5 seconds on this CPU.
- Later fixes: the frontend container got its own node_modules (Alpine and the host need different native binaries), and nginx now re-resolves the backend address after rebuilds (this fixed a 502 error).

Problems met

- Host PHP is 7.4, so PHP and Composer run only in containers.
- The queue worker crash-loops on a fresh setup until migrations have run; Docker restarts it automatically.

Phase 2: Database schema & ingestion

✓ **DONE**

Completed 2 Oct 2026

Goal: turn the official Pakistan Penal Code PDF into clean, searchable pieces (**chunks**), one per legal section, each with an embedding stored in PostgreSQL.

Steps completed

1. **Found the official source:** the Pakistan Penal Code PDF from pakistancode.gov.pk (179 pages; pages 1–29 are the table of contents, the law starts on page 30). Raw PDFs are not committed; the README has the download link.
2. **Tables:** sources (one row per act: title, short name, unit, year, URL, file checksum) and chunks (one row per section: chapter, section number, heading, text, token count, a vector(768) embedding, the model name). Indexes: **HNSW** for fast similarity search and **GIN** on a generated full-text column, kept for keyword search later.
3. **Text extraction** (DocumentTextExtractor). Plain PDF-to-text mixed amendment footnotes into the law text (“2Subs. by Ord. 21 of 1960...”) and stuck footnote numbers to words (“2[Pakistan]”). Solution: `pdftohtml -xml` gives the font size of every piece of text. Footnotes use a smaller font, so only body-size text is kept. Page footers (“Page 30 of 179”) are removed by pattern.
4. **Section-aware chunking** (LegalTextChunker). A new chunk starts at lines like 379., 381A. or [302.. The heading (“Punishment for theft”) and chapter are recorded. Sections longer than 500 tokens are split into overlapping windows of 50 tokens. Repeated stubs are dropped. A numbering check stops numbers inside sentences being mistaken for new sections.
5. **Ingestion service and command** `php artisan law:ingest`. It is **idempotent**: an unchanged file is skipped (checked by its SHA-256 checksum), and a changed file replaces its old chunks in one transaction.
6. **Embedding jobs** (EmbedChunks). Batches of 16 chunks go to the queue worker, which calls Ollama with the `search_document`: prefix the model was trained with. Each embedding's size is checked against the database column. `php artisan law:status` shows progress.
7. **Browser preview:** GET `/api/sources` and a search box on the React page.

Problems found in the real PDF and how they were fixed

Problem	Fix
Footnotes and footnote numbers mixed into the text	Keep only body-size text (font-size filter)
Newer poppler reports footers almost body size (17 vs 18 pt)	Remove “Page N of M” lines by pattern
Headings ending in “;”, “.--”, a quote (“Oath.”) or with no punctuation	Broader heading rules; stop where the rule text begins (“Whoever...”, “A person...”)
Doubled brackets “[[478.” and a missing space “[489A.Counterfeiting”	Section pattern accepts both (also recovered s.263A)
Greek question mark used as “;” and invisible soft hyphens	Characters normalised

Result

- **668 chunks covering 633 sections** (s.1–s.511 including lettered ones like 295C, 489F), every one with a heading.
- Only repeated sections are absent (e.g. s.18, s.325).
- Embedding all chunks takes about 13 minutes on the CPU.
- 21 automated tests at the end of the phase.

Phase 3: Retrieval, threshold & evaluation

✓ **DONE**

Completed 2 Oct 2026

Goal: given a question, reliably find the right sections, decide when *nothing* is relevant enough (so the assistant can refuse), and **measure** quality with numbers instead of guessing.

Steps completed

1. Retriever service:

- Embeds the question with the search_query: prefix and fetches 4× more candidates than needed.
- Keeps only the best window per section, so one long section can't fill the whole top 5.
- Splits results at the threshold into “relevant” and “below threshold”.
- Can be limited to specific acts; pgvector's iterative index scan stops filters returning too few rows.

2. `php artisan law:search "question"` prints the citations, headings and similarity scores for any question.

3. **Evaluation set** backend/resources/eval/ppc-retrieval.json: 33 questions in everyday language with the sections that count as correct, plus 10 off-topic questions (passport, khula, biryani...) that must be refused. Every expected section was checked against the database.

4. `php artisan law:eval` measures Hit@1, Hit@5 and MRR, and tries thresholds from 0.50 to 0.80 to find where answering and refusing are separated best.

5. **Threshold changed from 0.55 to 0.68** based on that sweep (0.55 would have answered the biryani question).

6. **Search API and page** now show a green “relevant sections found” or red “would refuse” banner, plus the closest matches below the threshold (greyed out).

7. **Tests:** 31 in total. The Retriever tests use hand-made vectors whose similarities are known exactly.

Measured results (baseline to beat)

Metric	Result	Meaning
Hit@1	23/33 (70%)	Correct section ranked first
Hit@5	28/33 (85%)	Correct section somewhere in the top 5 given to the LLM
MRR	0.756	Average of 1/rank (1.0 = always first)
In-scope top score	0.691 – 0.857	Similarity of the best match for real PPC questions
Off-topic top score	0.519 – 0.699	Similarity of the best match for unrelated questions
Threshold 0.68	33/33 answered, 9/10 refused	42 of 43 answer-or-refuse decisions correct

Main weakness found: vocabulary mismatch

The Penal Code uses legal and Arabic terms where people use everyday words. The misses follow that pattern: *murder* vs **qatl-i-amd** (s.302), *entering property* vs **criminal trespass** (s.441), *damaging property* vs **mischiefs** (s.425), *defending myself* vs **right of private defence** (s.96–100), *threatening for money* vs **extortion** (s.383). Phase 6 targets this.

Phase 4: Answer generation with citations

PLANNED

Up next

Goal: turn the retrieved sections into a short, plain-language answer written by the local LLM, where every claim cites its source as [1], [2]..., and refuse when nothing relevant was found.

What I will do

1. `OllamaClient::chat()`: call Ollama's `/api/chat` with a low temperature (≈ 0.1) for consistent, factual output, a timeout suited to CPU speed, and clear errors when Ollama is down.
2. `PromptBuilder`: a system prompt with the rules (answer only from the numbered sources, cite [n] after each claim, say so if the sources do not answer the question, never give legal advice), followed by the numbered sections (citation, heading, text) and the question. The prompt template is kept in `config/rag.php` so it can be tuned.
3. `CitationParser`: find [n] markers in the answer, map each to its section (e.g. [1] → PPC s.379), and **drop citations to numbers that were never given** (a hallucination guard).
4. `AnswerService`: coordinates the flow. Retrieve → if nothing is relevant, return the standard refusal without calling the LLM (fast and safe) → otherwise build the prompt → generate → parse citations.
5. queries **table**: log each question, answer, retrieved sections with scores, refused yes/no, model and response time. This data feeds Phase 8.
6. `POST /api/ask`: validated input; returns the answer, citations, source snippets, the disclaimer and timing. Returns 503 if Ollama is unavailable.
7. `php artisan law:ask "question"`: try answers from the terminal.
8. **Tests** with a fake Ollama (`Http::fake()`): prompt contents, citation mapping, removal of invalid citations, refusal path, error handling, query logging.

Done when

- Asking “What is the punishment for theft?” returns a short answer citing [1] PPC s.379.
- Asking an off-topic question returns the refusal message without calling the LLM.
- All tests pass; answers take roughly 5–15 seconds on this CPU.

What you will see / learn

- Prompt engineering for grounding, why the temperature is low, how to guard against invented citations.
- The full RAG loop working end to end.

Phase 5: Chat user interface

PLANNED
Not started

Goal: replace the search preview with a proper chat experience that makes the sources easy to check.

What I will do

1. **Chat window** with the message history (kept in React state), an input box and an example-question list.
2. **Answer bubbles** that render [1], [2] as clickable citation markers.
3. **Source panel:** clicking a citation shows the full section text, chapter and similarity score, highlighting which part supported the answer.
4. **Refusal display:** a clear “I can't answer this from the Penal Code” message that suggests rephrasing.
5. **Act filter** (useful once more laws are added in Phase 7).
6. **Loading and error states:** a typing indicator during the 5–15 second generation, and a friendly error if Ollama is down.
7. **Disclaimer** always visible; mobile-friendly layout.
8. **Frontend tests** with Vitest + React Testing Library for citation rendering and the API client.

Done when

- You can hold a conversation in the browser and click every citation to read the law it came from.

What you will see / learn

- Component design, typed API contracts between Laravel and React, accessible UI states.

Phase 6: Retrieval improvements

PLANNED
Not started

Goal: close the vocabulary gap found in Phase 3 and raise Hit@1 / Hit@5, proving every change with law:eval before and after.

What I will do

1. **Legal-term glossary / query expansion:** a reviewed list mapping everyday words to Penal Code terms (murder → qatl-i-amd, trespass → criminal trespass, damage → mischief, self-defence → private defence, bribe → gratification...). Matching terms are added to the question before embedding.
2. **Hybrid search:** combine vector similarity with PostgreSQL full-text search (the content_tsv column already exists) using Reciprocal Rank Fusion. This helps exact terms and section numbers ("section 302").
3. **Direct section lookup:** questions like "What does section 379 say?" go straight to that section.
4. **Optional re-ranker** (a small cross-encoder or LLM scoring) if the measured gain is worth the extra time.
5. **Grow the evaluation set** with new phrasings, keeping some questions aside so improvements aren't tuned only to the questions already seen (avoiding overfitting).
6. **Re-tune the threshold** after each change.

Done when

- Hit@1 and Hit@5 clearly above the Phase 3 baseline (70% / 85%), with off-topic refusals still working.
- A before/after table recorded in docs/architecture.md.

What you will see / learn

- Lexical vs semantic search, rank fusion, measuring before optimising, overfitting to an evaluation set.

Goal: make the assistant cover more than one act and prove the ingestion pipeline is general.

What I will do

1. **Constitution of the Islamic Republic of Pakistan, 1973:** download the official text, ingest with `--unit=article` so citations read "Constitution Art.25". Adapt the chunker if the article headings or schedules differ in format.
2. **Code of Criminal Procedure, 1898:** the procedure side (arrest, bail, FIR), which complements the PPC.
3. **Per-act inspection:** check section coverage, headings and footnote removal for each new PDF, as was done for the PPC.
4. **Evaluation sets** for each act, plus cross-act questions (e.g. "Can police arrest without a warrant for theft?").
5. **Act filter** in the API and UI; check the threshold still holds with more data.

Done when

- Three acts ingested and searchable, with citations in the right format (s. vs Art.) and evaluation numbers for each.

What you will see / learn

- How well a document pipeline generalises, and handling different document structures.

Phase 8: Answer-quality evaluation

PLANNED
Not started

Goal: measure the quality of the *answers*, not just retrieval, so the project can be discussed with evidence.

What I will do

1. **Answer evaluation set:** questions with the expected sections and key facts (e.g. theft → “up to three years, or fine, or both”).
2. **Automatic checks:** citation validity (every [n] points to a retrieved section), citation correctness (cites an expected section), refusal accuracy, and answer length.
3. **Faithfulness check:** does each sentence follow from its cited source? First a simple overlap check, then optionally an “LLM as judge” step, reported with its limits.
4. **Latency and resource report:** time per step (embedding, search, generation) on CPU; compare a larger model if the hardware allows.
5. php artisan law:eval-answers command and a results table in the docs.

Done when

- A results table: % answers with valid citations, % correct citations, refusal accuracy, average response time.

What you will see / learn

- How to evaluate LLM output, the trade-offs between quality and speed, and honest reporting of limits.

Phase 9: Engineering polish, CI & demo

PLANNED
Not started

Goal: make the repository look and behave like a professional project that is easy to review and demonstrate.

What I will do

1. **GitHub Actions CI:** on every push, run the backend tests (Postgres + pgvector service), the Pint style check, the frontend lint, type-check and build. Add a status badge to the README.
2. **Production-style Docker build:** the built React app served by nginx, PHP OPcache, config caching, no dev servers; a `docker-compose.prod.yml`.
3. **Optional: streaming answers** (Server-Sent Events) so text appears as it is generated.
4. **Security and robustness:** request rate limiting on `/api/ask`, input limits, no secrets in the repo, dependency checks.
5. **Documentation:** README screenshots or a GIF, a final architecture diagram, an evaluation summary, and a “what I learned” section.
6. **Demo script and interview notes:** a 5-minute walkthrough plus answers to likely questions (why pgvector, why local models, how hallucinations are prevented, how quality was measured).

Done when

- A green CI badge, a one-command demo, and a README that explains the project in under two minutes.

What you will see / learn

- CI/CD, production configuration, and presenting technical work clearly.

12. Key decisions

Decision	Why
pgvector inside PostgreSQL	One database for law text, metadata and vectors; SQL joins; no extra service to run.
Ollama local models	Free, private and offline. Trade-off: slower and less capable than cloud LLMs.
Laravel calls Ollama directly	The whole pipeline is visible in plain code, which is easier to explain than a framework.
One chunk per legal section	Citations point to exactly one provision; chunks are self-contained.
Font-size filtering of PDFs	Removes amendment footnotes reliably, where regex clean-up fails.
Relevance threshold + refusal	Grounding: never invent law when nothing relevant was found.
Measure with law:eval	Every tuning decision (threshold, chunking, search changes) is backed by numbers.
Queue for embeddings	Slow CPU work runs in the background and failed batches are retried.
PHP only in Docker	The host has PHP 7.4; Docker pins PHP 8.4 for everyone.

How to run the project

```
cd /var/www/html/learning/pakistan-law-assistant-rag-ai
make up                # start all containers
make ps                # all services should be "Up"
open http://localhost:5173 # React app: search + status
make artisan c="law:status" # ingested acts and embedding progress
make artisan c='law:search "punishment for theft"'
make artisan c="law:eval --details"
make test              # backend tests
```

Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation: find relevant passages first, then let the LLM answer only from them.
Embedding	A list of numbers (here 768) representing the meaning of a text; similar meanings give similar vectors.
Chunk	One stored piece of a law, here one section (or a window of a long section).
Cosine similarity	How closely two embeddings point in the same direction: 1 = same meaning, 0 = unrelated.
pgvector / HNSW	PostgreSQL extension for vectors / a graph index that finds nearest neighbours fast.
Threshold	Minimum similarity for a section to count as a source; below it the assistant refuses.
Hit@k	Share of questions where a correct section appears in the top k results.
MRR	Mean reciprocal rank: average of 1/rank of the first correct result.
Idempotent	Running an operation again gives the same result without duplicating data.
Hallucination	An LLM stating something not supported by its sources; citations make this checkable.